

12. Other Quality Attributes

Quality is not an act, it is a habit.

—Aristotle

Chapters 5–11 each dealt with a particular quality attribute important to software systems. Each of those chapters discussed how its particular quality attribute is defined, gave a general scenario for that quality attribute, and showed how to write specific scenarios to express precise shades of meaning concerning that quality attribute. And each gave a collection of techniques to achieve that quality attribute in an architecture. In short, each chapter presented a kind of portfolio for specifying and designing to achieve a particular quality attribute.

Those seven chapters covered seven of the most important quality attributes, in terms of their occurrence in modern software-reliant systems. However, as is no doubt clear, seven only begins to scratch the surface of the quality attributes that you might find needed in a software system you're working on.

Is cost a quality attribute? It is not a technical quality attribute, but it certainly affects fitness for use. We consider economic factors in Chapter 23.

This chapter will give a brief introduction to a few other quality attributes—a sort of “B list” of quality attributes—but, more important, show how to build the same kind of specification or design portfolio for a quality attribute not covered in our list.

12.1. Other Important Quality Attributes

Besides the quality attributes we've covered in depth in [Chapters 5–11](#), some others that arise frequently are variability, portability, development distributability, scalability and elasticity, deployability, mobility, and monitorability. We discuss “green” computing in [Section 12.3](#).

Variability

Variability is a special form of modifiability. It refers to the ability of a system and its supporting artifacts such as requirements, test plans, and configuration specifications to support the production of a set of variants that differ from each other in a preplanned fashion. Variability is an especially important quality attribute in a software product line (this will be explored in depth in [Chapter 25](#)), where it means the ability of a core asset to adapt to usages in the different product contexts that are within the product line scope. The goal of variability in a software product line is to make it easy to build and maintain products in the product line over a period of time. Scenarios for variability will deal with the binding time of the variation and the people time to achieve it.

Portability

Portability is also a special form of modifiability. Portability refers to the ease with which software that was built to run on one platform can be changed to run on a different platform. Portability is achieved by minimizing platform dependencies in the software, isolating dependencies to well-identified locations, and writing the software to run on a “virtual machine” (such as a Java Virtual Machine) that encapsulates all the platform dependencies within. Scenarios describing portability deal with moving software to a new platform by expending no more than a certain level of effort or by counting the number of places in the software that would have to change.

Development Distributability

Development distributability is the quality of designing the software to support distributed software development. Many systems these days are developed using globally distributed teams. One problem that must be overcome when developing with distributed teams is coordinating their activities. The system should be designed so that coordination among teams is minimized. This minimal coordination needs to be achieved both for the code and for the data model. Teams working on modules that communicate with each other may need to negotiate the interfaces of those modules. When a module is used by many other modules, each developed by a different team, communication and negotiation become more complex and burdensome. Similar considerations apply for the data model. Scenarios for development distributability will deal with the compatibility of the communication structures and data model of the system being developed and the coordination mechanisms of the organizations doing the development.

Scalability

Two kinds of scalability are horizontal scalability and vertical scalability. Horizontal scalability (scaling out) refers to adding more resources to logical units, such as adding another server to a cluster of servers. Vertical scalability (scaling up) refers to adding more resources to a physical unit, such as

adding more memory to a single computer. The problem that arises with either type of scaling is how to effectively utilize the additional resources. Being *effective* means that the additional resources result in a measurable improvement of some system quality, did not require undue effort to add, and did not disrupt operations. In cloud environments, horizontal scalability is called *elasticity*. Elasticity is a property that enables a customer to add or remove virtual machines from the resource pool (see [Chapter 26](#) for further discussion of such environments). These virtual machines are hosted on a large collection of upwards of 10,000 physical machines that are managed by the cloud provider. Scalability scenarios will deal with the impact of adding or removing resources, and the measures will reflect associated availability and the load assigned to existing and new resources.

Deployability

Deployability is concerned with how an executable arrives at a host platform and how it is subsequently invoked. Some of the issues involved in deploying software are: How does it arrive at its host (push, where updates are sent to users unbidden, or pull, where users must explicitly request updates)? How is it integrated into an existing system? Can this be done while the existing system is executing? Mobile systems have their own problems in terms of how they are updated, because of concerns about bandwidth. Deployment scenarios will deal with the type of update (push or pull), the form of the update (medium, such as DVD or Internet download, and packaging, such as executable, app, or plug-in), the resulting integration into an existing system, the efficiency of executing the process, and the associated risk.

Mobility

Mobility deals with the problems of movement and affordances of a platform (e.g., size, type of display, type of input devices, availability and volume of bandwidth, and battery life). Issues in mobility include battery management, reconnecting after a period of disconnection, and the number of different user interfaces necessary to support multiple platforms. Scenarios will deal with specifying the desired effects of mobility or the various affordances. Scenarios may also deal with variability, where the same software is deployed on multiple (perhaps radically different) platforms.

Monitorability

Monitorability deals with the ability of the operations staff to monitor the system while it is executing. Items such as queue lengths, average transaction processing time, and the health of various components should be visible to the operations staff so that they can take corrective action in case of potential problems. Scenarios will deal with a potential problem and its visibility to the operator, and potential corrective action.

Safety

In 2009 an employee of the Shushenskaya hydroelectric power station in Siberia sent commands over a network to remotely, and accidentally, activate an unused turbine. The offline turbine created a “water hammer” that flooded and then destroyed the plant and killed dozens of workers.

The thought that software could kill people used to belong in the realm of kitschy computers-run-amok science fiction. Sadly, it didn’t stay there. As software has come to control more and more of the

devices in our lives, software safety has become a critical concern.

Safety is not purely a software concern, but a concern for any system that can affect its environment. As such it receives mention in [Section 12.3](#), where we discuss system quality attributes. But there are means to address safety that are wholly in the software realm, which is why we discuss it here as well.

Software safety is about the software's ability to avoid entering states that cause or lead to damage, injury, or loss of life to actors in the software's environment, and to recover and limit the damage when it does enter into bad states. Another way to put this is that safety is concerned with the prevention of and recovery from hazardous failures. Because of this, the architectural concerns with safety are almost identical to those for availability, which is also about avoiding and recovering from failures. Tactics for safety, then, overlap with those for availability to a large degree. Both comprise tactics to prevent failures and to detect and recover from failures that do occur.

Safety is not the same as reliability. A system can be reliable (consistent with its specification) but still unsafe (for example, when the specification ignores conditions leading to unsafe action). In fact, paying careful attention to the specification for safety-critical software is perhaps the most powerful thing you can do to produce safe software. Failures and hazards cannot be detected, prevented, or ameliorated if the software has not been designed with them in mind. Safety is frequently engineered by performing failure mode and effects analysis, hazard analysis, and fault tree analysis. (These techniques are discussed in [Chapter 5](#).) These techniques are intended to discover possible hazards that could result from the system's operation and provide plans to cope with these hazards.

12.2. Other Categories of Quality Attributes

We have primarily focused on product qualities in our discussions of quality attributes, but there are other types of quality attributes that measure “goodness” of something other than the final product. Here are three:

Conceptual Integrity of the Architecture

Conceptual integrity refers to consistency in the design of the architecture, and it contributes to the understandability of the architecture and leads to fewer errors of confusion. Conceptual integrity demands that the same thing is done in the same way through the architecture. In an architecture with conceptual integrity, less is more. For example, there are countless ways that components can send information to each other: messages, data structures, signaling of events, and so forth. An architecture with conceptual integrity would feature one way only, and only provide alternatives if there was a compelling reason to do so. Similarly, components should all report and handle errors in the same way, log events or transactions in the same way, interact with the user in the same way, and so forth.

Quality in Use

ISO/IEC 25010, which we discuss in [Section 12.4](#), has a category of qualities that pertain to the use of the system by various stakeholders. For example, time-to-market is an important characteristic of a system, but it is not discernible from an examination of the product itself. Some of the qualities in this category are these:

- *Effectiveness*. This refers to the distinction between building the system correctly (the system performs according to its requirements) and building the correct system (the system performs in the manner the user wishes). Effectiveness is a measure of whether the system is correct.
- *Efficiency*. The effort and time required to develop a system. Put another way, what is the architecture’s impact on the project’s cost and schedule? Would a different set of architectural choices have resulted in a system that would be faster or cheaper to bring to fruition? Efficiency can include training time for developers; an architecture that uses technology unfamiliar to the staff on hand is less buildable. Is the architecture appropriate for the organization in terms of its experience and its available supporting infrastructure (such as test facilities or development environments)?
- *Freedom from risk*. The degree to which a product or system affects economic status, human life, health, or the environment.

A special case of efficiency is how easy it is to build (that is, compile and assemble) the system after a change. This becomes critical during testing. A recompile process that takes hours or overnight is a schedule-killer. Architects have control over this by managing dependencies among modules. If the architect doesn’t do this, then what often happens is that some bright-eyed developer writes a makefile early on, it works, and people add to it and add to it. Eventually the project ends up with a seven-hour compile step and very unhappy integrators and testers who are already behind schedule (because they always are).

Marketability

An architecture's marketability is another quality attribute of concern. Some systems are well known by their architectures, and these architectures sometimes carry a meaning all their own, independent of what other quality attributes they bring to the system. The current craze in building cloud-based systems has taught us that the perception of an architecture can be more important than the qualities the architecture brings. Many organizations have felt they had to build cloud-based systems (or some other *technology du jour*) whether or not that was the correct technical choice.

12.3. Software Quality Attributes and System Quality Attributes

Physical systems, such as aircraft or automobiles or kitchen appliances, that rely on software embedded within are designed to meet a whole other litany of quality attributes: weight, size, electric consumption, power output, pollution output, weather resistance, battery life, and on and on. For many of these systems, safety tops the list (see the sidebar).

Sometimes the software architecture can have a surprising effect on the system's quality attributes. For example, software that makes inefficient use of computing resources might require additional memory, a faster processor, a bigger battery, or even an additional processor. Additional processors can add to a system's power consumption, weight, required cabinet space, and of course expense.

Green computing is an issue of growing concern. Recently there was a controversy about how much greenhouse gas was pumped into the atmosphere by Google's massive processor farms. Given the daily output and the number of daily requests, it is possible to estimate how much greenhouse gas *you* cause to be emitted each time *you* ask Google to perform a search. (Current estimates range from 0.2 grams to 7 grams of CO₂.) Green computing is all the rage. Eve Troeh, on the American Public Media show "Marketplace" (July 5, 2011), reports:

Two percent of all U.S. electricity now goes to data centers, according to the Environmental Protection Agency. Electricity has become the biggest cost for processing data—more than the equipment to do it, more than the buildings to house that equipment. . . . Google's making data servers that can float offshore, cooled by ocean breezes. HP has plans to put data servers near farms, and power them with methane gas from cow pies.

The lesson here is that if you are the architect for software that resides in a larger system, you will need to understand the quality attributes that are important for the containing system to achieve, and work with the system architects and engineers to see how your software architecture can contribute to achieving them.

The Vanishing Line between Software and System Qualities

This is a book about software architecture, and so we treat quality attributes from a software architect's perspective. But you may have already noticed that the quality attributes that the software architect can bring to the party are limited by the architecture of the system in which the software runs.

For example:

- The performance of a piece of software is fundamentally constrained by the performance of the computer that runs it. No matter how well you design the software, you just can't run the latest whole-earth weather forecasting models on Grampa's Commodore 64 and hope to know if it's going to rain tomorrow.
- Physical security is probably more important and more effective than software security at preventing fraud and theft. If you don't believe this, write your laptop's password on a slip of paper, tape it to your laptop, and leave it in an unlocked car with the windows down. (Actually, don't really do that. Consider this a thought

experiment.)

- If we're being perfectly honest here, how usable is a device for web browsing that has a screen smaller than a credit card and keys the size of a raisin?

For me, nowhere is the barrier between software and system more nebulous than in the area of safety. The thought that software—strings of 0's and 1's—can kill or maim or destroy is still an unnatural notion. Of course, it's not the 0's and 1's that wreak havoc. At least, not directly. It's what they're connected to. Software, and the system in which it runs, has to be connected to the outside world in some way before it can do damage. That's the good news. The bad news is that the good news isn't all that good. Software *is* connected to the outside world, always. If your program has no effect whatsoever that is observable outside of itself, it probably serves no purpose.

There are notorious examples of software-related failures. The Siberian hydroelectric plant catastrophe mentioned in the text, the Therac-25 fatal radiation overdose, the Ariane 5 explosion, and a hundred lesser known accidents all caused harm because the *software* was part of a *system* that included a turbine, an X-ray emitter, or a rocket's steering controls, in the examples just cited. In these cases, flawed software commanded some hardware in the system to take a disastrous action, and the hardware simply obeyed. Actuators are devices that connect hardware to software; they are the bridge between the world of 0's and 1's and the world of motion and control. Send a digital value to an actuator (or write a bit string in the hardware register corresponding to the actuator) and that value is translated to some mechanical action, for better or worse.

But connection to an actuator is not required for software-related disasters. Sometimes all the computer has to do is send erroneous information to its human operators. In September 1983, a Soviet satellite sent data to its ground system computer, which interpreted that data as a missile launched from the United States aimed at Moscow. Seconds later, the computer reported a second missile in flight. Soon, a third, then a fourth, and then a fifth appeared. Soviet Strategic Rocket Forces lieutenant colonel Stanislav Yevgrafovich Petrov made the astonishing decision to ignore the warning system, believing it to be in error. He thought it extremely unlikely that the U.S. would have fired just a few missiles, thereby inviting total retaliatory destruction. He decided to wait it out, to see if the missiles were real—that is, to see if his country's capital city was going to be incinerated. As we know, it wasn't. The Soviet system had mistaken a rare sunlight condition for missiles in flight. Similar mistakes have occurred on the U.S. side.

Of course, the humans don't always get it right. On the dark and stormy night of June 1, 2009, Air France flight 447 from Rio de Janeiro to Paris plummeted into the Atlantic Ocean, killing all on board. The Airbus A-330's flight recorders were not recovered until May 2011, and as this book goes to publication it appears that the pilots never knew that the aircraft had entered a high-altitude stall. The sensors that measure airspeed had become clogged with ice and therefore unreliable. The software was

required to disengage the autopilot in this situation, which it did. The human pilots thought the aircraft was going too fast (and in danger of structural failure) when in fact it was going too slow (and falling). During the entire three-minute-plus plunge from 38,000 feet, the pilots kept trying to pull the nose up and throttles back to lower the speed. It's a good bet that adding to the confusion was the way the A-330's stall warning system worked. When the system detects a stall, it emits a loud audible alarm. The computers deactivate the stall warning when they "think" that the angle of attack measurements are invalid. This can occur when the airspeed readings are very low. That is exactly what happened with Air France 447: Its forward speed dropped below 60 knots, and the angle of attack was extremely high. As a consequence of a rule in the flight control *software*, the stall warning stopped and started several times. Worse, it came on whenever the pilot let the nose fall a bit (increasing the airspeed and taking the readings into the "valid" range, but still in stall) and then stopped when he pulled back. That is, doing the right thing resulted in the wrong feedback and vice versa.

Was this an unsafe system, or a safe system unsafely operated? Ultimately the courts will decide.

Software that can physically harm us is a fact of our modern life. Sometimes the link between software and physical harm is direct, as in the Ariane example, and sometimes it's much more tenuous, as in the Air France 447 example. But as software professionals, we cannot take refuge in the fact that our software can't actually inflict harm any more than the person who shouts "Fire!" in a crowded theater can claim it was the stampede, not the shout, that caused injury.

—PCC

12.4. Using Standard Lists of Quality Attributes—or Not

Architects have no shortage of lists of quality attributes for software systems at their disposal. The standard with the pause-and-take-a-breath title of “ISO/IEC FCD 25010: Systems and software engineering—Systems and software product Quality Requirements and Evaluation (SQuaRE)—System and software quality models,” is a good example. The standard divides quality attributes into those supporting a “quality in use” model and those supporting a “product quality” model. That division is a bit of a stretch in some places, but nevertheless begins a divide-and-conquer march through a breathtaking array of qualities. See [Figure 12.1](#) for this array.

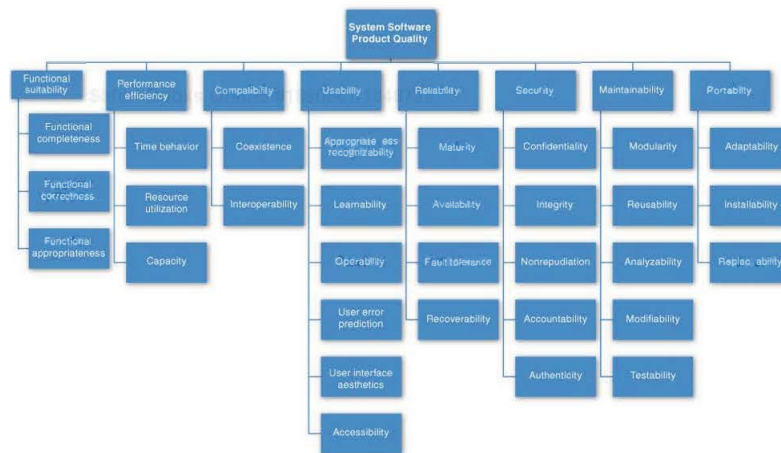


Figure 12.1. The ISO/IEC FCD 25010 product quality standard

The standard lists the following quality attributes that deal with product quality:

- *Functional suitability.* The degree to which a product or system provides functions that meet stated and implied needs when used under specified conditions
- *Performance efficiency.* Performance relative to the amount of resources used under stated conditions
- *Compatibility.* The degree to which a product, system, or component can exchange information with other products, systems, or components, and/or perform its required functions, while sharing the same hardware or software environment
- *Usability.* The degree to which a product or system can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use
- *Reliability.* The degree to which a system, product, or component performs specified functions under specified conditions for a specified period of time
- *Security.* The degree to which a product or system protects information and data so that persons or other products or systems have the degree of data access appropriate to their types and levels of authorization

- *Maintainability*. The degree of effectiveness and efficiency with which a product or system can be modified by the intended maintainers
- *Portability*. The degree of effectiveness and efficiency with which a system, product, or component can be transferred from one hardware, software, or other operational or usage environment to another

In ISO 25010, these “quality characteristics” are each composed of “quality subcharacteristics” (for example, nonrepudiation is a subcharacteristic of security). The standard slogs through almost five dozen separate descriptions of quality subcharacteristics in this way. It defines for us the qualities of “pleasure” and “comfort.” It distinguishes among “functional correctness” and “functional completeness,” and then adds “functional appropriateness” for good measure. To exhibit “compatibility,” systems must either have “interoperability” or just plain “coexistence.” “Usability” is a product quality, not a quality-in-use quality, although it includes “satisfaction,” which *is* a quality-in-use quality. “Modifiability” and “testability” are both part of “maintainability.” So is “modularity,” which is a strategy for achieving a quality rather than a goal in its own right. “Availability” is part of “reliability.” “Interoperability” is part of “compatibility.” And “scalability” isn’t mentioned at all.

Got all that?

Lists like these—and there are many—do serve a purpose. They can be helpful checklists to assist requirements gatherers in making sure that no important needs were overlooked. Even more useful than standalone lists, they can serve as the basis for creating your own checklist that contains the quality attributes of concern in your domain, your industry, your organization, and your products. Quality attribute lists can also serve as the basis for establishing measures. If “pleasure” turns out to be an important concern in your system, how do you measure it to know if your system is providing enough of it?

However, general lists like these also have drawbacks. First, no list will ever be complete. As an architect, you will be called upon to design a system to meet a stakeholder concern not foreseen by any list-maker. For example, some writers speak of “manageability,” which expresses how easy it is for system administrators to manage the application. This can be achieved by inserting useful instrumentation for monitoring operation and for debugging and performance tuning. We know of an architecture that was designed with the conscious goal of retaining key staff and attracting talented new hires to a quiet region of the American Midwest. That system’s architects spoke of imbuing the system with “Iowability.” They achieved it by bringing in state-of-the-art technology and giving their development teams wide creative latitude. Good luck finding “Iowability” in any standard list of quality attributes, but that QA was as important to that organization as any other.

Second, lists often generate more controversy than understanding. You might argue persuasively that “functional correctness” should be part of “reliability,” or that “portability” is just a kind of “modifiability,” or that “maintainability” is a kind of “modifiability” (not the other way around). The writers of ISO 25010 apparently spent time and effort deciding to make security its own characteristic, instead of a subcharacteristic of functionality, which it was in a previous version. We believe that effort in making these arguments could be better spent elsewhere.

Third, these lists often purport to be *taxonomies*, which are lists with the special property that every

member can be assigned to exactly one place. Quality attributes are notoriously squishy in this regard. We discussed denial of service as being part of security, availability, performance, and usability in [Chapter 4](#).

Finally, these lists force architects to pay attention to every quality attribute on the list, even if only to finally decide that the particular quality attribute is irrelevant to their system. Knowing how to quickly decide that a quality attribute is irrelevant to a specific system is a skill gained over time.

These observations reinforce the lesson introduced in [Chapter 4](#) that quality attribute names, by themselves, are largely useless and are at best invitations to begin a conversation; that spending time worrying about what qualities are subqualities of what other qualities is also almost useless; and that scenarios provide the best way for us to specify precisely what we mean when we speak of a quality attribute.

Use standard lists of quality attributes to the extent that they are helpful as checklists, but don't feel the need to slavishly adhere to their terminology.

12.5. Dealing with “X-ability”: Bringing a New Quality Attribute into the Fold

Suppose, as an architect, you must deal with a quality attribute for which there is no compact body of knowledge, no “portfolio” like [Chapters 5–11](#) provided for those seven QAs? Suppose you find yourself having to deal with a quality attribute like “green computing” or “manageability” or even “Iowability”? What do you do?

Capture Scenarios for the New Quality Attribute

The first thing to do is interview the stakeholders whose concerns have led to the need for this quality attribute. You can work with them, either individually or as a group, to build a set of attribute characterizations that refine what is meant by the QA. For example, security is often decomposed into concerns such as confidentiality, integrity, availability, and others. After that refinement, you can work with the stakeholders to craft a set of specific scenarios that characterize what is meant by that QA.

Once you have a set of specific scenarios, then you can work to generalize the collection. Look at the set of stimuli you’ve collected, the set of responses, the set of response measures, and so on. Use those to construct a general scenario by making each part of the general scenario a generalization of the specific instances you collected.

In our experience, the steps described so far tend to consume about half a day.

Assemble Design Approaches for the New Quality Attribute

After you have a set of guiding scenarios for the QA, you can assemble a set of design approaches for dealing with it. You can do this by

1. Revisiting a body of patterns you’re familiar with and asking yourself how each one affects the QA of interest.
2. Searching for designs that have had to deal with this QA. You can search on the name you’ve given the QA itself, but you can also search for the terms you chose when you refined the QA into subsidiary attribute characterizations (such as “confidentiality” for the QA of security).
3. Finding experts in this area and interviewing them or simply writing and asking them for advice.
4. Using the general scenario to try to catalog a list of design approaches to produce the responses in the response category.
5. Using the general scenario to catalog a list of ways in which a problematic architecture would fail to produce the desired responses, and thinking of design approaches to head off those cases.

Model the New Quality Attribute

If you can build a conceptual model of the quality attribute, this can be helpful in creating a set of design approaches for it. By “model,” we don’t mean anything more than understanding the set of parameters to which the quality attribute is sensitive. For example, a model of modifiability might tell us that modifiability is a function of how many places in a system have to be changed in response to a

modification, and the interconnectedness of those places. A model for performance might tell us that throughput is a function of transactional workload, the dependencies among the transactions, and the number of transactions that can be processed in parallel.

Once you have a model for your QA, then you can work to catalog the architectural approaches (tactics and patterns) open to you for manipulating each of the relevant parameters in your favor.

Assemble a Set of Tactics for the New Quality Attribute

There are two sources that can be used to derive tactics for any quality attribute: models and experts.

Figure 12.2 shows a queuing model for performance. Such models are widely used to analyze the latency and throughput of various types of queuing systems, including manufacturing and service environments, as well as computer systems.

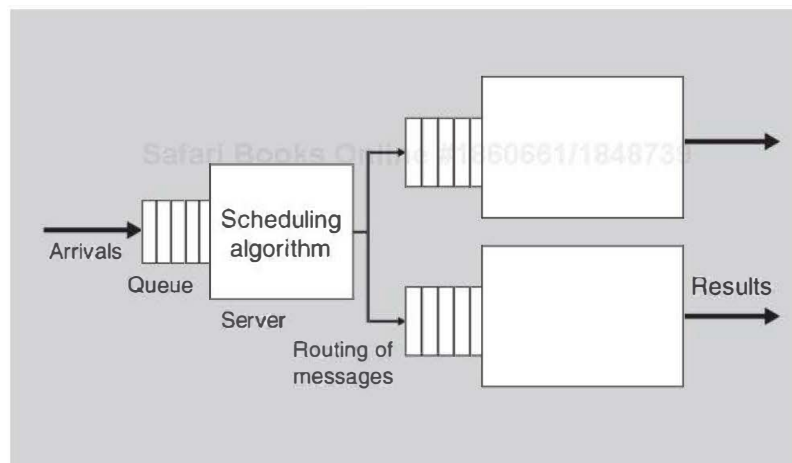


Figure 12.2. A generic queuing model

Within this model, there are seven parameters that can affect the latency that the model predicts:

- Arrival rate
- Queuing discipline
- Scheduling algorithm
- Service time
- Topology
- Network bandwidth
- Routing algorithm

These are the *only* parameters that can affect latency within this model. This is what gives the model its power. Furthermore, each of these parameters can be affected by various architectural decisions. This is what makes the model useful for an architect. For example, the routing algorithm can be fixed or it could be a load-balancing algorithm. A scheduling algorithm must be chosen. The topology can be

affected by dynamically adding or removing new servers. And so forth.

The process of generating tactics based on a model is this:

- Enumerate the parameters of the model
- For each parameter, enumerate the architectural decisions that can affect this parameter

What results is a list of tactics to, in the example case, control performance and, in the more general case, to control the quality attribute that the model is concerned with. This makes the design problem seem much more tractable. This list of tactics is finite and reasonably small, because the number of parameters of the model is bounded, and for each parameter, the number of architectural decisions to affect the parameter is limited.

Deriving tactics from models is fine as long as the quality attribute in question has a model. Unfortunately, the number of such models is limited and is a subject of active research. There are no good *architectural* models for usability or security, for example. In the cases where we had no model to work from, we did four things to catalog the tactics:

1. We interviewed experts in the field, asking them what they do as architects to improve the quality attribute response.
2. We examined systems that were touted as having high usability (or testability, or whatever tactic we were focusing on).
3. We scoured the relevant design literature looking for common themes in design.
4. We examined documented architectural patterns to look for ways they achieved the quality attribute responses touted for them.

Construct Design Checklists for the New Quality Attribute

Finally, examine the seven categories of design decisions in [Chapter 4](#) and ask yourself (or your experts) how to specialize your new quality of interest to these categories. In particular, think about reviewing a software architecture and trying to figure out how well it satisfies your new qualities in these seven categories. What questions would you ask the architect of that system to understand how the design attempts to achieve the new quality? These are the basis for the design checklist.

12.6. For Further Reading

For most of the quality attributes we discussed in this chapter, the Internet is your friend. You can find reasonable discussions of scalability, portability, and deployment strategies using your favorite search engine. Mobility is harder to find because it has so many meanings, but look under “mobile computing” as a start.

Distributed development is a topic covered in the International Conference on Global Software Engineering, and looking at the proceedings of this conference will give you access to the latest research in this area (www.icgse.org).

Release It! [Nygard 07] has a good discussion of monitorability (which he calls transparency) as well as potential problems that are manifested after extended operation of a system. The book also includes various patterns for dealing with some of the problems.

To gain an appreciation for the importance of software safety, we suggest reading some of the disaster stories that arise when software fails. A venerable source is the ACM Risks Forum newsgroup, known as comp.risks in the USENET community, available at www.risks.org. This list has been moderated by Peter Neumann since 1985 and is still going strong.

Nancy Leveson is an undisputed thought leader in the area of software and safety. If you’re working in safety-critical systems, you should become familiar with her work. You can start small with a paper like [Leveson 04], which discusses a number of software-related factors that have contributed to spacecraft accidents. Or you can start at the top with [Leveson 11], a book that treats safety in the context of today’s complex, sociotechnical, software-intensive systems.

The Federal Aviation Administration is the U.S. government agency charged with oversight of the U.S. airspace system, and the agency is extremely concerned about safety. Their 2000 System Safety Handbook is a good practical overview of the topic [FAA 00].

IEEE STD-1228-1994 (“Software Safety Plans”) defines best practices for conducting software safety hazard analyses, to help ensure that requirements and attributes are specified for safety-critical software [IEEE 94]. The aeronautical standard DO-178B (due to be replaced by DO-178C as this book goes to publication) covers software safety requirements for aerospace applications.

A discussion of safety tactics can be found in the work of Wu and Kelly [Wu 06].

In particular, interlocks are an important tactic for safety. They enforce some safe sequence of events, or ensure that a safe condition exists before an action is taken. Your microwave oven shuts off when you open the door because of a hardware interlock. Interlocks can be implemented in software also. For an interesting case study of this, see [Wozniak 07].

12.7. Discussion Questions

1. The Kingdom of Bhutan measures the happiness of its population, and government policy is formulated to increase Bhutan's GNH (gross national happiness). Go read about how the GNH is measured (try www.grossnationalhappiness.com) and then sketch a general scenario for the quality attribute of *happiness* that will let you express concrete happiness requirements for a software system.
2. Choose a quality attribute not described in [Chapters 5–11](#). For that quality attribute, assemble a set of specific scenarios that describe what you mean by it. Use that set of scenarios to construct a general scenario for it.
3. For the QA you chose for discussion question 2, assemble a set of design approaches (patterns and tactics) that help you achieve it.
4. For the QA you chose for discussion question 2, develop a design checklist for that quality attribute using the seven categories of guiding quality design decisions outlined in [Chapter 4](#).
5. What might cause you to add a tactic or pattern to the sets of quality attributes already described in [Chapters 5–11](#) (or any other quality attribute, for that matter)?
6. According to slate.com and other sources, a teenage girl in Germany “went into hiding after she forgot to set her Facebook birthday invitation to private and accidentally invited the entire Internet. After 15,000 people confirmed they were coming, the girl’s parents canceled the party, notified police, and hired private security to guard their home.” Fifteen hundred people showed up anyway; several minor injuries ensued. Is Facebook “unsafe”? Discuss.
7. Author James Gleick (“A Bug and a Crash,” www.around.com/ariane.html) writes that “It took the European Space Agency 10 years and \$7 billion to produce Ariane 5, a giant rocket capable of hurling a pair of three-ton satellites into orbit with each launch. . . All it took to explode that rocket less than a minute into its maiden voyage. . . was a small computer program trying to stuff a 64-bit number into a 16-bit space. One bug, one crash. Of all the careless lines of code recorded in the annals of computer science, this one may stand as the most devastatingly efficient.” Write a safety scenario that addresses the Ariane 5 disaster and discuss tactics that might have prevented it.
8. Discuss how you think development distributability tends to “trade off” against the quality attributes of performance, availability, modifiability, and interoperability.
Extra Credit: Close your eyes and, without peeking, spell “distributability.” Bonus points for successfully saying “development distributability” three times as fast as you can.
9. What is the relationship between mobility and security?
10. Relate monitorability to observability and controllability, the two parts of testability. Are they the same? If you want to make your system more of one, can you just optimize for the other?